

Alexander Batrakov - Ausgewähltes Projektportfolio

Data Science, Data Analytics, Machine Learning und Software Engineering

Wuppertal, Germany | batrakovalex.astro@gmail.com | github.com/AlexBatrakov | linkedin.com/in/alexbatrakov97

Portfolio-Zusammenfassung

Dieses Portfolio ergänzt meinen einseitigen Lebenslauf um sechs ausgewählte Projekte in Data Science, Data Analytics, Machine Learning und reproduzierbaren Analyse- und Modellierungsworkflows. Die Projekte zeigen, wie ich mit unordentlichen Daten, analytischem SQL, ML-Evaluation, backendgestützten Experiment-Workflows, Simulationsumgebungen und wissenschaftlicher Softwareentwicklung arbeite.

Einordnung des Portfolios

Die Projekte sind bewusst unterschiedlich und decken komplementäre Fähigkeiten in Analytics, ML, Backend-Workflows, Simulation und wissenschaftlicher Software ab. Der Abschnitt **Rollenrelevante Nachweise** ordnet Anforderungen den Nachweisen zu; **Projektübersicht** und Projektkarten zeigen konkrete Umsetzungen.

Rollenrelevante Nachweise

Kompetenzbereich	Worauf zu achten ist	Projekte
Datenqualität & SQL-Analytics	Bereinigte Pipelines, Qualitätslabels, Data Dictionary, Artefaktberichte, DuckDB/PostgreSQL-Marts, zeitbewusste Joins, Rolling Windows und analytische Views.	Wearable Analytics
ML-Experimente & Modellzuverlässigkeit	Multi-Seed-Auswahl, kontrollierte Ablationen, MLflow/Artefakt-Tracking, Kalibrierung, Fehleranalyse, Feature Selection, Baselines, Action-Masks und konservative Holdouts.	PyTorch Pets Wearable Solo Wargame AI
Backend/API & Run-Orchestrierung	FastAPI-Services, datenbankgestützter Run-Zustand, SQLAlchemy/Alembic, Redis/RQ-Worker, normalisierte Configs, Run-Attempts, Metriken/Artefakte, Retries, Docker/CI-Setups.	Evalynx PyTorch Pets
Statistische Diagnostik	Residuenstatistiken, normalisierte Zusammenfassungen, Konvergenzmetriken, Normalitätschecks, White-Noise-Fits, adaptive Grids, Priors und Kandidaten-Reranking.	GravityToolsNext Wearable
Simulation & AI-Agenten	Deterministischer Simulator, gestufte Entscheidungslogik, Hidden-Information-Regeln, Legal-Action-Masken, Replay-Traces, Agentenfamilien, Exact/Reference-Workflows und Fixed-Seed-Benchmarks.	Solo Wargame AI
Scientific Computing	Julia-Package-APIs, ODE-Solver, Shooting-Methoden, Equation-of-State-Abstraktionen (EoS), Sensitivitätsdiagnostik, strukturierte Run-Artefakte, Docs, CI und Tests.	GravityToolsNext StructureSolver
Reproduzierbarkeit & Engineering	Tests, Docs, CI, MLflow, lokal nachvollziehbare CLI-/Docker-Setups, deterministisches Replay, Datenverträge, maschinenlesbare Artefakte und dokumentierte Annahmen.	Alle Projekte

Projektübersicht

Projekt	Primäres Signal	Rolle im Portfolio
Wearable Analytics	DA / DS / Time Series	Qualitätsbewusster Garmin-Analytics-Workflow mit SQL-Marts, Feature Engineering und leakage-bewusster Modellierung.
PyTorch Pets Classifier	ML / Reliability / Deployment	Reproduzierbarer PyTorch-CV-Workflow mit MLflow-Tracking, Diagnostik, Kalibrierung und FastAPI/Docker-Deployment.
Evalynx	Backend / Experiment Platform	FastAPI/PostgreSQL/Redis-Control-Plane für Computing-Runs, Async Worker, Retries, Metriken und Artefakte.
Solo Wargame AI	Simulation / AI-Agenten / Evaluation	Deterministischer Hidden-Information-Simulator mit Legal-Action-Masken, Replay, Fixed-Seed-Benchmarks und Agentenfamilien.
GravityToolsNext.jl	Statistische Diagnostik / RSE	Julia-Framework für externe Model-Fitting-Workflows mit strukturierten Metriken, Residualdiagnostik, Priors und adaptiver Parametersuche.
StructureSolver.jl	Numerical Computing / Julia	Julia-Package für ODE-Simulationen, Shooting-Methoden, Parameterscans und Sensitivitätsanalyse.

Rollensignale: [Data Analytics](#) [Data Science](#) [Time Series](#) [SQL](#) [ML Evaluation](#)**Kurzprofil:** Privacy-first Garmin-Analytics-Workflow, der unordentliche JSON/FIT-Exporte in bereinigte Parquet-Datasets, Qualitätslabels, SQL-Marts, statistische Validierung, Monitoring-Features und leakage-bewusste Next-Sleep-Modelle überführt.**Umgesetzt**

- Implementierte eine gestufte lokale Pipeline für Garmin-JSON/FIT-Exporte: Discovery, Ingestion, Bereinigung, Parquet-Checkpoints, Data Dictionary, Qualitätslabels und Suspicious-Day-Reports.
- Baute DuckDB/PostgreSQL-Analytics-Layer mit Views, CTE/Window Queries, day-to-next-sleep-Joins, Rolling-Recovery-Metriken, Stress-Quartilen, Risk Flags und modellierungsfertigen SQL-Extrakten.
- Entwickelte Monitoring- und sleep-aligned Feature-Tabellen aus minutengenauen HR/Stress-Daten, inklusive semantischer Wake/Sleep-Fenster, Coverage/Gap-Qualitätsfeatures und Prior-History/State-Context-Prädiktoren.
- Entwickelte leakage-bewusste Modellierungsworkflows mit Feature-Set-Vergleichen, Feature Selectors, Clipping/Kalibrierung, Dummy-Baselines, Random/Temporal-Validierung, Finalisten-Refits und Holdout-Diagnostik.

Ausgewählte Nachweise

- 677 Tageszeilen und 1,56 Mio.+ minutengenaue HR/Stress-Beobachtungen aus lokalen Garmin-Exporten verarbeitet.
- 617 / 677 Tage als strict-good gelabelt; 21 korrupte stress-only Artefakttage vor Analyse/Modellierung markiert.
- DuckDB/PostgreSQL-SQL-Layer zeigt day-to-next-sleep-Joins, Rolling Windows, Quartile, Risk Flags und modellierungsfertige Extrakte.
- 52.812 lineare Modellkonfigurationen ausgewertet; validierungsselektiertes Huber-Modell reduzierte Fixed-Future-Holdout-MAE um 15,8% gegenüber Dummy-Median; 150 lokale Tests bestanden.

Nachweis für: Datenqualität, SQL-Analytics, Analytics Engineering, Time-Series Feature Engineering, leakage-bewusste Validierung, reproduzierbare Modellierungsworkflows und konservative DS/ML-Evaluation.**Stack:** Python, pandas, NumPy, SciPy, scikit-learn, DuckDB, PostgreSQL, parquet, Typer CLI, pytest, GitHub Actions

PyTorch Pets Classifier

Rollensignale: [Machine Learning](#) [Computer Vision](#) [ML Evaluation](#) [Model Reliability](#) [Deployment](#)**Kurzprofil:** Reproduzierbarer PyTorch-Computer-Vision-Workflow mit ResNet18 Transfer Learning, kontrollierten Ablationen, MLflow-gestütztem Experiment-Tracking, Three-Seed-Modellauswahl, strukturierter Fehleranalyse, Kalibrierungsschecks und FastAPI/Docker/Azure-Deployment-Pfad.**Umgesetzt**

- Implementierte eine konfigurationsgetriebene PyTorch/torchvision Trainings- und Evaluationspipeline für ResNet18 Transfer Learning auf 37 Oxford-IIIT Pets Klassen, mit Early Stopping, Scheduler-Optionen, Top-k-Metriken, Checkpoint-Metadaten und checkpoint-bewusster Inferenz.
- Baute einen reproduzierbaren Experimentworkflow mit YAML-Configs, isolierten Run-Artefakten, MLflow-Logging, Vergleichstabellen, Seed-Family-Summaries, öffentlichen Experimentberichten und Makefile-Kommandos.
- Ergänzte Diagnostik jenseits von Accuracy: Per-Class-Metriken, wichtigste Confusion-Pairs, Per-Sample-Predictions, Confidence-Histogramme, overconfident errors, Error-Delta-Vergleich und Kalibrierungsschecks.
- Verpackte den selektierten Checkpoint hinter einem getesteten FastAPI-Inferenzservice mit Docker, Azure Container Apps Deployment, GitHub Actions CI/CD, gepinnten Model-Manifest, Checksum-Verifikation und Smoke-Checks.

Ausgewählte Nachweise

- 21 Experimentkonfigurationen mit kontrollierten Vergleichen über Scheduler, Resolution, Weight Decay, Batch/LR, Freeze, Augmentation und Seed-Sweep dokumentiert.
- Showcase-Rezept nach Three-Seed-Robustheit statt Single-Best-Run ausgewählt; Test Top-1 Accuracy: 88,29% ± 0,15%.
- Strukturierte Fehleranalyse- und Reliability-Reports für Per-Class-Performance, Confusion-Pairs, Confidence-Verhalten, Kalibrierungsmetriken und Before/After Error Deltas.
- Modell über FastAPI, Docker, Azure Container Apps, GitHub Actions CI/CD, gepinnte Release-Artefakte und Post-Deploy Smoke-Checks verfügbar gemacht.
- Lokale Suite: 62 Tests bestanden für API-Verhalten, Prediction, Transforms, Tracking, Kalibrierung, Experiment Analytics und Error-Delta-Logik.

Nachweis für: kontrollierte ML-Experimente, MLflow-gestütztes Tracking, robustheitsbewusste Modellauswahl, Modell-diagnostik, Kalibrierung, Experiment Analytics, PyTorch Transfer Learning und Model-to-API Deployment.**Stack:** Python, PyTorch, torchvision, ResNet18, MLflow, FastAPI, Docker, Azure Container Apps, GitHub Actions, pytest, Makefile

Rollensignale: [Backend](#) [API](#) [Async Jobs](#) [Experiment Platform](#) [Software Engineering](#)

Kurzprofil: FastAPI/PostgreSQL/Redis Backend-Control-Plane für reproduzierbare Computing-Runs, das Ad-hoc-Skripte und verstreute Logs in API-gesteuerte Run-Submission, datenbankgestützten Lifecycle-Zustand, asynchrone Ausführung, Retry-Historie und strukturierte Ergebnispersistenz überführt.

Umgesetzt

- Implementierte ein FastAPI-Backend mit Project/Run-Endpunkten, Service/Repository/Worker-Layern, SQLAlchemy/Alembic und PostgreSQL-gestütztem Run-State.
- Modellerte dauerhafte Ausführungshistorie mit Project, Run und RunAttempt-Datensätze, inklusive normalisierter Configs, Config-Hashes, Lifecycle-Status, Metriken, Artefakten, Warnings/Errors und Timestamps.
- Ergänzte Redis/RQ-gestützte asynchrone Ausführung mit workerseitiger Attempt-Verarbeitung, retry-sicherem Failed-Run-Handling, Stale-Attempt-Replay-Schutz und Artefaktverzeichnissen pro Attempt.
- Integrierte externe Berechnung über begrenzte Runner-Adapter und einen Subprocess-JSON-Contract; paketierte einen lokales Reviewer-Setup mit Docker Compose und GitHub Actions CI Smoke Tests.

Ausgewählte Nachweise

- 8 API-Endpunkte für Health, Projektmanagement, Run Submission, Run Listing/Detail, projektbezogene Runs und Failed-Run-Retry.
- SQLAlchemy/Alembic-Modell mit Project, Run und RunAttempt, plus Migrationen für Result Fields und Attempt Backfill.
- Redis/RQ-Ausführungspfad mit Retry-Semantik, der fehlgeschlagene Attempts erhält und aktuelle Run-Snapshots vor stale workers schützt.
- Docker-Compose-Stack mit api, worker, postgres, redis und migrate services; öffentlicher Reviewer kann die Stub-Demo lokal ausführen.
- GitHub Actions validiert pytest, Modulkompilierung, Docker-Stack-Build, Migrationen, Service-Start und HTTP-Smoke-Tests; 29 pytest Tests.

Nachweis für: Backend-API-Design, relationales State Modeling, Async Job Orchestration, Retry/Failure-Semantik, reproduzierbare Experimentausführung, Docker-Compose-Packaging und CI-gestützte Software-Engineering-Workflows.

Stack: Python, FastAPI, SQLAlchemy, Alembic, PostgreSQL, Redis, RQ, Pydantic, Docker Compose, GitHub Actions, pytest

Solo Wargame AI

Rollensignale: [Simulation](#) [AI-Agenten](#) [ML Evaluation](#) [Research Software](#) [Software Engineering](#)

Kurzprofil: Deterministischer Python-Simulations- und Policy-Evaluations-Stack für stochastische Hidden-Information-Entscheidungen, der Rulebook-Logik in testbare State/Action-Contracts, replayfähige Episoden, Fixed-Seed-Benchmarks und Agent-Evaluationsworkflows überführt.

Umgesetzt

- Übersetzte ein regeltextlastiges stochastisches Regelsystem in explizite Python-Domain-Contracts: State, Actions, gestufte Entscheidungskontexte, Legal-Action-Generierung, Transitions, Observations, Terminalzustände und Validierung.
- Implementierte deterministische RNG-Snapshots, Replay-Traces, striktes TOML-Mission-Parsing/Validation, Hidden-Information-Regeln, Gym-style Environment Wrappers, feste Action-Kataloge und policy-facing Legality Masks.
- Baute Evaluationsoberflächen für random, heuristic, rollout-search, exact-guided und PyTorch masked actor-critic Agents, mit festen Benchmark-Seeds und getrennten Train/Model-Selection/Evaluation-Seed-Rollen.
- Ergänzte reproduzierbares Tooling um den Simulator: CLI-Kommandos, JSON Episode Batch Runner, SQLite Exact/Policy-Audit-Artefakte, maschinenlesbare Metriken, lokales Browser Play, pytest/Ruff-Verifikation und CI.

Ausgewählte Nachweise

- Layered Python Package mit deterministischem Simulator, Mission Configs, Legal-Action Engine, Replay, Gym-style Wrappers, Agents, CLIs, lokalem Browser Play und Tests.
- Fixed-Seed Mission-1-Benchmark: random 11/200, learned best 144/200, heuristic 157/200, rollout-search 195/200; exact fair ceiling etwa 0,95.
- Learned masked actor-critic bleibt explizit als Agentenfamilie enthalten; Abstände zu den Baselines werden sichtbar gemacht statt versteckt.
- Maschinenlesbare JSON-Evaluationsworkflows enthalten Request/Response-Schemas, Ausführungsmetadaten, aggregierte Metriken und optionale JSONL-Episodenzeilen.
- Verifikation: 356 Tests bestanden; Ruff Check sauber.

Nachweis für: Simulation Engineering, AI-Agent-Evaluation, deterministisches Testen, Legal-Action Masking, baseline-kontrollierte Experimente, Replay/Debugging und reproduzierbare Evaluationsworkflows.

Stack: Python, PyTorch, Gym-style wrappers, TOML, JSON/JSONL, SQLite, pytest, Ruff, CLI tooling, stdlib HTTP server

Rollensignale: [Statistische Diagnostik](#) [Experiment-Orchestrierung](#) [Scientific Computing](#) [Research Software](#) [Julia](#)

Kurzprofil: Julia-Framework für reproduzierbare externe Model-Fitting-Workflows, das Run-Artefakte in strukturierte Metriken, Residualdiagnostik, White-Noise-Fits, priorbasierte Analysen, adaptive Grids und getestete Docs/Docker-Oberflächen überführt.

Umgesetzt

- Entwarf typisierte Julia-Settings und deterministische Job-Materialisierung für externe TEMPO/TEMPO2-Runs, inklusive validierter Pfade, staged Inputs/Outputs, Workspace Policies, Output Capture, Cleanup Rules und strukturierter Artefakte.
- Implementierte Parser und Result Objects für Tempo2-stdout, par/tim-Dateien, Residual-Dateien, Parameterschätzungen, interne Iterationen, Konvergenzzusammenfassungen, Fit-Metriken und Failure States.
- Baute statistische Diagnostik-Layer für Residuenstatistiken, normalisierte Summaries, Backend-Gruppierung, Konvergenzmetriken, Normalitätstests und backend-aware White-Noise EFAC/EQUAD/offset fitting.
- Entwickelte prior-aware und adaptive Parametersuch-Workflows mit analytic/grid/sampled Priors, posteriorgewichtete Aggregation, JLD2-Persistenz und rein Julia-basierte adaptive 2D-Grids für teure Objective Surfaces.

Ausgewählte Nachweise

- Wandelt rohe External-Run-Artefakte in strukturierte Result Objects mit Chi-square, weighted RMS, normalisierten Residual-Summaries, Konvergenzstatus, Parameterschätzungen und Pre/Post-Fit-Deltas um.
- Implementiert Residual- und White-Noise-Diagnostikmodule mit backend-aware Fit Objects, EFAC/EQUAD/Offset-Konzepte, Normalitätsdiagnostik und Anderson-Darling-artigen Objective-Support.
- Kombiniert prior-marginalisierte Ausführung und adaptive Grid-Suche für Parameterexploration über teure Model-Fitting Surfaces.
- Öffentliches Repository mit Documenter Docs, Beispielen, CI Workflows, Docker-Entwicklungsbaseline und breiter Julia Test Suite.

Nachweis für: statistische Diagnostik, Residuenanalyse, uncertainty-aware Model Evaluation, adaptive Parametersuche, externe Model-Fitting-Orchestrierung, strukturierte Metriкеxtraktion und reproduzierbare Julia-Workflows.

Stack: Julia, TEMPO/TEMPO2, HypothesisTests, Distributions, KernelDensity, Optim, StatsBase, JLD2, Documenter.jl, Docker

StructureSolver.jl

Rollensignale: [Scientific Computing](#) [Numerical Methods](#) [Sensitivity Analysis](#) [Research Software](#) [Julia](#)

Kurzprofil: Julia-Package für ODE-basierte Simulationsworkflows mit Equation-of-State-Abstraktionen (EoS), Shooting-Methoden, Parameterscans, ForwardDiff-Sensitivitätsdiagnostik, Tests und Docs.

Umgesetzt

- Entwarf ein modulares Julia-Package mit Trennung von EoS-Backends, physikalischen Modellen, Solver-Regimes, Simulationstreibern, Utilities, Dokumentation und Tests.
- Implementierte direkte und Shooting-Simulationsregimes für ODE-basierte Solves, inklusive Integrationskontrollen, Boundary-Condition-Zielen, strukturierter Quantity Extraction und Named Outputs.
- Baute Scan- und Sensitivitätsworkflows für Single Solves, Family/Grid-Scans, N-D-Shooting-Scans, ForwardDiff-basierte Ableitungen und NLSolve-basiertes Boundary Matching.

Ausgewählte Nachweise

- Implementiert wiederverwendbare EoS-, Model-, Regime-, Simulation-, Shooting-, Scan- und Sensitivity-Analysis-Interfaces statt einmaliger numerischer Skripte.
- Überführt Domäneneingaben in dokumentierte Named Outputs und Diagnostik wie Massen/Radien, boundary residuals, central quantities, sound speed und Ableitungsausgaben.
- Öffentliches Julia-Package mit Documenter Docs, Beispielen, CI Workflows, Release-/Zitationsmetadaten und lokaler Verifikation: 69 / 69 Tests bestanden.

Nachweis für: numerisches Scientific Computing, Julia-Package-Design, ODE-Simulationsworkflows, Shooting-Methoden, Parameterscans, Sensitivitätsanalyse und dokumentierte Output-Definitionen.

Stack: Julia, DifferentialEquations.jl, ForwardDiff.jl, NLSolve.jl, Interpolations.jl, Documenter.jl, GitHub Actions

Gemeinsamer Arbeitsstil

Über die Projekte hinweg lege ich Wert auf explizite Datenverträge, reproduzierbare Runs, Baseline-Vergleiche, statistische Diagnostik, testbare Komponenten, dokumentierte Annahmen und prüfbare Artefakte. Das wiederkehrende Muster ist, Inputs sichtbar zu machen, Zwischenergebnisse zu erhalten, gegen Baselines zu vergleichen und Workflows nachvollziehbar und überprüfbar zu halten.